MyCodeSchool

Code review — sebastian-one-user-management

Runde de review pe proiectul tău, cea mai nouă sus.

Review pe proiectul `sebastian-one-user-management`: poarta de compilare, `mvn test` și verificare prin rulare pe H2 (copie în afara repo-ului tău). Codul tău nu a fost modificat.

Runda 1 — 2026-10-20 — sha 937897f

Poarta de compilare trece, `mvn test` are 0 eșecuri și 0 erori. Cele 3 constatări 🔴 de mai jos au fost **reproduse prin rulare**, nu deduse. B1 și B2 au aceeași cauză: `sort` vine din exterior și nu e filtrat.

🔴 Critice

B1 🔴 Sortarea după `password` e permisă: oricine poate afla ordinea parolelor

`UserController.java:120`

CUM E ACUM: `GET /api/users/search?sort=password,asc` răspunde 200 și întoarce utilizatorii exact în ordinea alfabetică a parolelor: Radu (parola000), Maria (parola111), Cristian (parola123), Bogdan (parola456), Ana (parola789). Parola nu apare niciodată în JSON, dar ordinea o trădează.

DE CE: `Pageable` din `@PageableDefault` primește `sort` direct din query string, iar Spring Data îl transformă în `ORDER BY` pe ORICE atribut al entității `User`, inclusiv `password`. `UserResponse` ascunde câmpul la ieșire, dar sortarea are loc în SQL, înainte de mapare, deci ascunderea din DTO nu protejează nimic. Cu un cont creat de el cu o parolă aleasă, un atacator vede unde cade în listă și află între ce valori sunt parolele vecinilor. În rulare, contul `Spion` cu `parola200` a ieșit între Cristian (parola123) și Bogdan (parola456). Repetat cu alte parole, asta devine o căutare binară. Pentru că parolele sunt stocate în clar, ordinea e chiar ordinea parolelor reale.

FIX: restrânge proprietățile sortabile la o listă albă (`lastName`, `firstName`, `age`, `email`) înainte de a trimite `Pageable` în repository: verifici `pageable.getSort()` și răspunzi 400 dacă apare altă proprietate.

Repro:

```
curl -s 'localhost:18093/api/users/search?sort=password,asc&size=10'
-> 200: Radu, Maria, Cristian, A, Spion, Bogdan, Ana
```

Contul `Spion` e creat înainte cu `POST {"password":"parola200",...}` -> 201 și apare între parola123 și parola456.

+ același bug la:

- `UserController.java:247` (`/api/users/from-age?age=1&sort=password,asc` , reprodus: aceeași ordine)
- `UserQueryServiceImpl.java:52` (`search`) și `:93` (`getUsersFromAge`): pasează `Pageable` -ul necurățat către repository; locul bun pentru validarea sortului, dacă nu o faci în controller
- `UserRepository.java:52` (`findByAgeGreaterThanEqual(int, Pageable)`) și `:69` (`@Query search(..., Pageable)`): consumă sortul din `Pageable` ca `ORDER BY` ; sunt singurele 2 query-uri paginate din repo
- Nu mai există alte locuri cu `Pageable` / `Sort` controlat de client: celelalte listări au ordine fixă în cod (`findAllByOrderByLastNameAscFirstNameAsc` , `findTop3ByOrderByAgeDesc` , `findByLastNameOrderByFirstNameAsc`), iar `CatalogRunner.java:48` folosește `Sort.by("age")` hardcodat

B2🔴 `sort` pe un câmp inexistent dă 500, nu 400

`GlobalExceptionHandler.java:24`

CUM E ACUM: `GET /api/users/search?sort=foo` -> 500 Internal Server Error, cu corpul implicit Spring (fără `message` , fără `details`). Log: `InvalidDataAccessApiUsageException: UnknownPathException: Could not resolve attribute 'foo' of 'ro.mycode.user_management.users.models.User'` .

DE CE: numele din `sort` nu e verificat când se construiește `Pageable` . Spring îl pune ca text în obiectul `Sort` . Hibernate abia încearcă să-l rezolve când construiește interogarea, în repository, adică după ce cererea a trecut de controller și de validare. Excepția se transformă în `InvalidDataAccessApiUsageException` , pentru care `GlobalExceptionHandler` nu are niciun `@ExceptionHandler` , așa că ajunge la `/error` și pleacă cu 500. Clientul a greșit (400), dar serverul pare că a căzut. B1 și B2 au aceeași cauză: `sort` vine din exterior și nu e filtrat.

FIX: lista albă de la B1 rezolvă și asta: proprietatea necunoscută răspunde 400 cu `ApiError` înainte să ajungă în repository.

Repro:

```
curl -s -w '%{http_code}' 'localhost:18093/api/users/search?sort=foo'
-> {"status":500,"error":"Internal Server Error",
    "path":"/api/users/search"} 500
curl 'localhost:18093/api/users/from-age?sort=foo' -> 500
```

+ același bug la:

- `UserController.java:120` (`/search`) și `:247` (`/from-age`): singurele două endpoint-uri cu `Pageable` ; ambele dau 500 la `sort=foo` (reprodus)
- `UserRepository.java:52` și `:69` : punctele unde ajunge `Sort` -ul clientului și aruncă excepția; fix-ul stă într-un whitelist în controller sau într-un `@ExceptionHandler` pentru excepția de proprietate necunoscută -> 400
- Aceeași familie, validare neacoperită: `search?size=-1` și `search?page=abc` dau 200 (Spring Data le ignoră silențios și folosește valorile implicite), nu 400 cu `ApiError` ; același loc, `UserController.java:120` și `:247`
- Nu sunt afectate: `/adults` , `/older-than` , `/between-ages` , `/by-last-name` , `/by-email` (nu au `Pageable`)

B3 🔴 Nume mai lung de 50 de caractere -> 409 Conflict în loc de 400

`UserCreateRequest.java:16`

CUM E ACUM: POST cu `firstName` de 51 de caractere -> 409, cu mesajul `Request violates a database constraint` și `details: []` . PATCH cu `lastName` de 51 de caractere pe un utilizator existent -> tot 409. Clientul nu află ce câmp e greșit, iar 409 înseamnă în documentația API-ului „emailul e deja folosit".

DE CE: limita de 50 există doar în `@Column(length = 50)` (`User.java:32` , `:37`). Asta e o informație pentru DDL, nu o regulă de validare. Bean Validation nu o citește, deci `@Valid` din controller trece și cererea ajunge până la INSERT/UPDATE. Abia acolo H2 refuză: `Value too long for column "FIRST_NAME CHARACTER VARYING(50)"` (în log). Spring traduce codul SQL 22001 în `DataIntegrityViolationException` , iar handler-ul de la `GlobalExceptionHandler.java:99` răspunde 409 la orice excepție de acest tip. Deci cererea e respinsă corect, dar prea târziu și cu statusul altei erori. La PATCH e și mai înșelător: eroarea apare la flush, la commit, după ce serviciul a terminat deja.

FIX: `@Size(max = 50)` pe `firstName` / `lastName` și `@Size(max = 100)` pe `email` , în `UserCreateRequest` și `UserUpdateRequest` . Atunci răspunsul e 400 cu `details: ["firstName: ..."]` .

Repro:

```
curl -X POST -H 'Content-Type: application/json' \
  -d '{"firstName":"<51 x a>","lastName":"X",
       "email":"long1@x.com","password":"parola123","age":20}' \
  localhost:18093/api/users
-> 409 {"message":"Request violates a database constraint",
        "details":[]}
curl -X PATCH -d '{"lastName":"<51 x a>"}' .../api/users/<id Maria>
-> 409
```

+ același bug la:

- `UserCreateRequest.java:19-20` (`lastName` , fără `@Size(max = 50)` ; reprodus: 409)
- `UserCreateRequest.java:24-27` (`email` : are `@Email` , dar nu `@Size(max = 100)` ; reprodus cu un email valid de 110 caractere -> 409)
- `UserUpdateRequest.java:14-15` (`firstName`) și `:17-18` (`lastName`): reprodus pe PATCH -> 409
- `UserUpdateRequest.java:20-23` (`email`): același mecanism, nereprodus separat pe PATCH
- `User.java:30-38` și `:41-43` : entitatea are `@NotBlank` , dar nu `@Size` ; nu e vizibil în API, fiindcă DTO-urile validează primele, dar ar prinde depășirea înainte de flush
- `GlobalExceptionHandler.java:99-103` : 409 generic pentru orice `DataIntegrityViolationException` , deci maschează cauza reală; fix-ul principal e validarea în DTO-uri, nu aici
- Fără alte cazuri: parola are doar `min = 8` (coloana nu are `length` explicit), `age` e validat cu `@Positive` / `@Max`

🟡 Importante

- **M1** `User.java:50` + `UserRepository.java:33` : parola e stocată în clar, iar `findByEmailAndPassword` compară text cu text. Ăsta e motivul pentru care B1 scoate la iveală parolele reale. Hash cu BCrypt când ajungi la Security.
- **M2** `GlobalExceptionHandler.java:24` : 405 (`PUT /api/users/{id}`), 415 (POST cu `text/plain`) și 500 ies cu corpul implicit Spring, fără `message` / `details` , nu ca `ApiError` . Reprodus: 405 și 415. Contractul „o singură formă de eroare" se rupe.
- **M3** `GlobalExceptionHandler.java:92` : POST fără `age` -> 400 `Malformed JSON request` , deși JSON-ul e valid. În Jackson 3, `FAIL_ON_NULL_FOR_PRIMITIVES` e activ pe `int age` , iar mesajul nu spune ce câmp lipsește. Reprodus.
- **M4** `UserCreateRequest.java:38` : `"age": 1.9` e acceptat și salvat ca 1 (201). `int` primește float trunchiat. Reprodus.

- **M5** `application.yaml:4-5` : `root/root` și URL-ul MySQL sunt scrise direct în fișier. Convenția e `${DB_USERNAME}` / `${DB_PASSWORD}` din env vars.
- **M6** `UserController.java:280` : `/exists?email=` -> 200 `{"email":"","exists":false}` , fără validare, spre deosebire de `/by-email` , care are `@NotBlank @Email` . Reprodus.

🟢 Cleanups

- **C1** `UserRepository.java:31` , `:39` , `:59` : `deleteByEmail` , `findByEmailNative` , `searchByName` nu sunt folosite de niciun endpoint, doar de demo (`CatalogRunner`) sau deloc.
- **C2** `application.yaml:11` : `show-sql: true` + `format_sql` în profilul implicit umple logul în producție.
- **C3** `UserController.java:120` : `page=-1` e transformat în tăcere în pagina 0 (reprodus), nu respins.

🟢 Ce e bine făcut

- Poarta de compilare trece, iar `mvn test` are 0 eșecuri și 0 erori.
- Corpul `ApiError` e unitar pe 400/404/409, iar `details` sunt sortate și au numele câmpului.
- POST întoarce 201 cu header `Location` , iar PUT pe `/{id}` dă corect 405.
- `@Modifying(clearAutomatically = true, flushAutomatically = true)` pe `updatePasswordByEmail` e corect, nu lasă entitatea veche în context.
- `equals` / `hashCode` pe entitate sunt făcute corect: id nenul + `getClass().hashCode()` .
- `open-in-view: false` și `@Transactional(readOnly = true)` pe serviciul de citire.
- Semantica PATCH (null sau gol = nemodificat) e documentată și respectată.

Before / After (pentru 🔴 , doar în document, nu aplicat în cod)

B1 + B2 — lista albă pe `sort`

Before (acum)	After (cum ar trebui)
`Pageable` ajunge necurățat în service și repository; `sort=password` merge, `sort=foo` dă 500	verifici proprietățile din `Sort` înainte de apel; orice altceva -> 400 cu `ApiError`

Before:

```
@GetMapping("/search")
public Page<UserResponse> search(
    @RequestParam String q,
    @PageableDefault(size = 10) Pageable pageable) {
  return service.search(q, pageable);
}
```

After:

```
private static final Set<String> SORTABLE =
    Set.of("lastName", "firstName", "age", "email");

private void checkSort(Pageable pageable) {
  for (Sort.Order o : pageable.getSort()) {
    if (!SORTABLE.contains(o.getProperty())) {
      throw new InvalidSortException(o.getProperty());
    }
  }
}
```

`InvalidSortException` îl prinzi în `GlobalExceptionHandler` și răspunzi 400 cu `ApiError`, ca la celelalte erori de validare.

B3 — limita de lungime în DTO

Before (acum)	After (cum ar trebui)
limita 50 doar în `@Column`; cererea ajunge la INSERT și dă 409	`@Size(max = 50)` în DTO; cererea e respinsă la controller cu 400

Before:

```
@NotBlank
private String firstName;
```

After:

```
@NotBlank
@Size(max = 50)
private String firstName;
```

La fel pentru `lastName` (50) și `email` (100), în `UserCreateRequest` și `UserUpdateRequest`.

Q&A

1. De ce B1 e o scurgere de informații chiar dacă `UserResponse` nu are câmpul `password`? În ce moment al cererii are loc sortarea și în ce moment ascunderea?
2. `@Column(length = 50)` și `@Size(max = 50)` par să spună același lucru. Cine citește fiecare dintre ele și când? Explică pe asta de ce ai primit 409, nu 400.
3. Care e drumul exact al lui `?sort=foo` de la URL până la 500? Unde ai putea opri cererea ca să iasă 400 cu `ApiError`?